

Testovací scénáře - Veselský Jan

Odpovědnost: Vehicle Entity + IR01-IR04 (State, Dispatcher, Async, Router)

Část 1: Business Entita - Vehicle (Stavový automat)

TC-V1: Inicializace vozidla s výchozím stavem DRAFT

Vstup: `new Vehicle({brand: 'Škoda', model: 'Octavia', year: 2022})`

Očekávaný výstup:

- `vehicle.status === 'DRAFT'`
- `vehicle.id` je vygenerováno
- `vehicle.createdAt` je nastaveno

TC-V2: Validní přechod DRAFT → AVAILABLE (admin)

Vstup:

- Vehicle se stavem `DRAFT`
- `userRole = 'admin'`

Akce: `vehicle.updateStatus('AVAILABLE', 'admin')`

Očekávaný výstup:

- `{success: true, vehicle: {...}}`
- `vehicle.status === 'AVAILABLE'`
- `vehicle.updatedAt` aktualizováno

TC-V3: Nevalidní přechod DRAFT → RENTED (nepovolený přechod)

Vstup: Vehicle se stavem `DRAFT`

Akce: `vehicle.updateStatus('RENTED', 'admin')`

Očekávaný výstup:

- `{success: false, error: 'Nepovolený přechod: DRAFT → RENTED'}`
- Stav zůstává `DRAFT`

TC-V4: Přechod AVAILABLE → RENTED (systémově při aktivaci rezervace)

Vstup: Vehicle se stavem `AVAILABLE`

Akce: `vehicle.updateStatus('RENTED', 'system')`

Očekávaný výstup:

- `{success: true, vehicle: {...}}`
- `vehicle.status === 'RENTED'`

TC-V5: Invariant - vozidlo ve stavu RENTED nelze vyřadit

Vstup: Vehicle se stavem `RENTED`

Akce: `vehicle.canDeleteOrDecommission()`

Očekávaný výstup:

- `{success: false, error: 'Vozidlo je aktuálně vypůjčeno - nelze smazat ani vyřadit'}`

TC-V6: Přechod AVAILABLE → MAINTENANCE (admin)

Vstup: Vehicle se stavem `AVAILABLE`

Akce: `vehicle.updateStatus('MAINTENANCE', 'admin')`

Očekávaný výstup:

- `{success: true, vehicle: {...}}`
- `vehicle.status === 'MAINTENANCE'`

TC-V7: Přechod MAINTENANCE → AVAILABLE (servis dokončen)

Vstup: Vehicle se stavem `MAINTENANCE`

Akce: `vehicle.updateStatus('AVAILABLE', 'admin')`

Očekávaný výstup:

- `{success: true, vehicle: {...}}`
- `vehicle.status === 'AVAILABLE'`

TC-V8: Přejchod MAINTENANCE → DECOMMISSIONED (nerentabilní oprava)

Vstup: Vehicle se stavem `MAINTENANCE`

Akce: `vehicle.updateStatus('DECOMMISSIONED', 'admin')`

Očekávaný výstup:

- `{success: true, vehicle: {...}}`
- `vehicle.status === 'DECOMMISSIONED'`

TC-V9: Přejchod AVAILABLE → DECOMMISSIONED (prodej vozu)

Vstup: Vehicle se stavem `AVAILABLE`

Akce: `vehicle.updateStatus('DECOMMISSIONED', 'admin')`

Očekávaný výstup:

- `{success: true, vehicle: {...}}`
- `vehicle.status === 'DECOMMISSIONED'`

TC-V10: Invariant - stav DECOMMISSIONED je terminální

Vstup: Vehicle se stavem `DECOMMISSIONED`

Akce: `vehicle.canTransitionTo('AVAILABLE')`

Očekávaný výstup: `false`

TC-V11: Invariant tachometru - nelze snížit stav

Vstup: Vehicle s `mileage: 50000`

Akce: `vehicle.updateMileage(45000)`

Očekávaný výstup:

- `{success: false, error: 'Stav tachometru nesmí klesnout: 50000 → 45000'}`
- `vehicle.mileage` zůstává 50000

TC-V12: Validní aktualizace tachometru

Vstup: Vehicle s `mileage: 50000`

Akce: `vehicle.updateMileage(55000)`

Očekávaný výstup:

- `{success: true, vehicle: {...}}`
- `vehicle.mileage === 55000`
- `vehicle.updatedAt` aktualizováno

TC-V13: Invariant - pouze admin může měnit stav na MAINTENANCE

Vstup: Vehicle se stavem `AVAILABLE`

Akce: `vehicle.updateStatus('MAINTENANCE', 'employee')`

Očekávaný výstup:

- `{success: false, error: 'Nedostatečná práva pro tuto operaci'}`

TC-V14: Kontrola dostupnosti pro rezervaci

Vstup: Vehicle se stavem `AVAILABLE`

Akce: `vehicle.checkAvailability()`

Očekávaný výstup: `true`

TC-V15: Kontrola nedostupnosti pro rezervaci (MAINTENANCE)

Vstup: Vehicle se stavem `MAINTENANCE`

Akce: `vehicle.checkAvailability()`

Očekávaný výstup: `false`

Část 2: IR01 – State Management

TC-S1: Inicializace stavu aplikace

Vstup: Volání `getState()` na začátku

Očekávaný výstup:

```
{
  vehicles: { byId: {}, allIds: [], loading: false, error: null },
  reservations: { byId: {}, allIds: [], loading: false, error: null },
  ui: { currentView: 'vehicles', ... },
  auth: { user: null, isAuthenticated: false, ... }
}
```

TC-S2: Mutace ADD_VEHICLE přidá vozidlo do stavu

Vstup: Počáteční stav s prázdnými `vehicles`

Akce: `mutate({ type: 'ADD_VEHICLE', payload: {id: 'v1', brand: 'BMW'} })`

Očekávaný výstup:

- `state.vehicles.byId['v1']` obsahuje data vozidla
- `state.vehicles.allIds` obsahuje `'v1'`

TC-S3: Mutace UPDATE_VEHICLE aktualizuje existující vozidlo

Vstup: Stav s vozidlem `v1` (brand: 'BMW')

Akce: `mutate({ type: 'UPDATE_VEHICLE', payload: {id: 'v1', brand: 'Audi'} })`

Očekávaný výstup:

- `state.vehicles.byId['v1'].brand === 'Audi'`
- `state.vehicles.byId['v1'].updatedAt` je aktualizováno

TC-S4: Mutace REMOVE_VEHICLE odstraní vozidlo

Vstup: Stav s vozidlem `v1`

Akce: `mutate({ type: 'REMOVE_VEHICLE', payload: 'v1' })`

Očekávaný výstup:

- `state.vehicles.byId['v1']` je undefined
- `state.vehicles.allIds` neobsahuje `'v1'`

TC-S5: Subscribery jsou notifikováni o změnách stavu

Vstup: Registrovaný subscriber callback

Akce: `mutate({ type: 'ADD_VEHICLE', payload: {...} })`

Očekávaný výstup: Subscriber callback je vyvolán s novým stavem

TC-S6: State je immutable - kopie není reference

Vstup: `const state1 = getState()`

Akce: `mutate({ type: 'ADD_VEHICLE', ... })`, pak `const state2 = getState()`

Očekávaný výstup: `state1 !== state2` (různé reference)

Část 3: IR02 - Dispatcher

TC-D1: Registrace handleru pro akci

Vstup: Prázdný dispatcher

Akce: `registerAction('TEST_ACTION', handler)`

Očekávaný výstup: Handler je uložen v `actionHandlers`

TC-D2: Dispatch validní akce

Vstup: Registrovaný handler pro 'FETCH_VEHICLES'

Akce: `dispatch({ type: 'FETCH_VEHICLES' })`

Očekávaný výstup:

- Handler je vyvolán
- Vrácen `{ success: true, data: [...] }`

TC-D3: Dispatch nevalidní akce (bez handleru)

Vstup: Neexistující typ akce

Akce: `dispatch({ type: 'UNKNOWN_ACTION' })`

Očekávaný výstup:

- `{ success: false, error: 'No handler for action: UNKNOWN_ACTION' }`

TC-D4: Dispatch akce bez typu

Vstup: Akce bez `type`

Akce: `dispatch({ payload: 'data' })`

Očekávaný výstup:

- `{ success: false, error: 'Invalid action' }`

TC-D5: Dispatcher volá business logiku (Vehicle.updateStatus)

Vstup: Akce `UPDATE_VEHICLE_STATUS` s nevalidním přechodem

Akce: `dispatch({ type: 'UPDATE_VEHICLE_STATUS', payload: {...} })`

Očekávaný výstup:

- Volá `vehicle.updateStatus()`
 - Pokud neúspěch: `{ success: false, error: ... }`
 - Notifikace o chybě přidána do stavu
-

Část 4: IR03 – Asynchronní operace

TC-A1: FETCH_VEHICLES načte data z Mock API

Vstup: Prázdný stav

Akce: `dispatch({ type: 'FETCH_VEHICLES' })`

Očekávaný výstup:

- `state.vehicles.loading === true` (během požadavku)
- Po dokončení: `state.vehicles.loading === false`
- `state.vehicles.byId` obsahuje načtená data

TC-A2: Chyba API je zachycena a uložena

Vstup: Simulovaná síťová chyba

Akce: `dispatch({ type: 'FETCH_VEHICLES' })` při selhání sítě

Očekávaný výstup:

- `state.vehicles.error === 'Síťová chyba...'`
- `state.vehicles.loading === false`

TC-A3: CREATE_VEHICLE uloží data přes API

Vstup: Data nového vozidla

Akce: `dispatch({ type: 'CREATE_VEHICLE', payload: {...} })`

Očekávaný výstup:

- Volání `mockApiCall('POST', '/vehicles', ...)`
- Po úspěchu: Notifikace "Vozidlo bylo vytvořeno"
- Vozidlo přidáno do stavu

TC-A4: DELETE_VEHICLE smaže data přes API

Vstup: Existující vozidlo `v1`

Akce: `dispatch({ type: 'DELETE_VEHICLE', payload: {vehicleId: 'v1'} })`

Očekávaný výstup:

- Volání `mockApiCall('DELETE', '/vehicles/v1')`
 - Po úspěchu: Vozidlo odstraněno ze stavu
-

Část 5: IR04 – Router

TC-R1: Parse URL pro vehicles

Vstup: `window.location.hash = '#/vehicles'`

Akce: `handleRouteChange()`

Očekávaný výstup:

- `{ view: 'vehicles', title: 'Vozidla', params: {}, path: '/vehicles' }`
- Dispatch `NAVIGATE` s `view: 'vehicles'`

TC-R2: Parse URL pro detail vozidla

Vstup: `window.location.hash = '#/vehicles/v123'`

Akce: `handleRouteChange()`

Očekávaný výstup:

- `{ view: 'vehicle-detail', params: {id: 'v123'}, ... }`
- Dispatch `NAVIGATE` s `vehicleId: 'v123'`

TC-R3: Programatická navigace změny URL

Vstup: Aktuální URL `#/vehicles`

Akce: `navigateTo('#/reservations')`

Očekávaný výstup: `window.location.hash === '#/reservations'`

TC-R4: Router reaguje na popstate

Vstup: Registrovaný event listener

Akce: Uživatel klikne "zpět" v prohlížeči

Očekávaný výstup: `handleRouteChange()` je vyvolán

TC-R5: Synchronizace URL ze stavu

Vstup: `state.ui.currentView = 'reservations'`

Akce: `updateUrlFromState(state)`

Očekávaný výstup: `window.location.hash === '#/reservations'`

Shrnutí pokrytí

Kategorie	Počet testů	Pokrytí
Vehicle FSM	15	Všechny stavy, přechody, invarianty
IR01 State	6	Inicializace, mutace, subscribery
IR02 Dispatcher	5	Registrace, dispatch, chyby
IR03 Async	4	API volání, loading, error stavy
IR04 Router	5	URL parsing, navigace, synchronizace
CELKEM	35	Kompletní pokrytí odpovědnosti